IT5014 - C# and .NET PROGRAMMING

Assignment - Inheritance and ADO .NET Database Connectivity using C# .NET

Submitted by

Name	**Yuvaraj K**
Register Number	**2022506079**

Department of Information Technology
Madras Institute of Technology,
Anna University, Chennai – 600 044

August 2024 – December 2024

Contents

1 Inheritance 1

1.1 Definition 1

1.2 Implementation in C# 1

1.3 Characteristics of Inheritance 1

1.4 Benefits of Inheritance 1

1.5 Interface 2

1.6 Types of Inheritance 2

1.7 Classical Inheritance 2

1.7.1 Single Inheritance 3

1.7.2 Multiple Inheritance 5

1.7.3 Multilevel Inheritance 7

1.7.4 Hierarchical Inheritance 10

1.7.5 Hybrid Inheritance 13

1.8 Container Inheritance 17

2 ADO .NET Database Connectivity 19

2.1 Introduction 19

2.2 Connected Architecture 19

2.2.1 .NET Data Providers 20

2.2.2 Sample Code and Output 20

2.3 Disconnected Architecture 23

2.3.1 Sample Code and Output 23

1 Inheritance

1.1 Definition

Inheritance is a core object-oriented programming concept. It is the mechanism of basing a class upon another class, i.e., constructing or deriving a new class from an existing class.

The new class is called as the child class or sub class or derived class. It is a more specific class. The existing class is called as the parent class or super class or base class. It is a more general class.

1.2 Implementation in C#

In C#, inheritance is implemented in 2 ways.

1. Inheriting classes, i.e., extending super classes
2. Implementing interfaces, i.e., implementing super interfaces.

1.3 Characteristics of Inheritance

1. A class can’t inherit more than one super class.
2. A class can inherit more than one super interface.
3. Sub classes can access only those members of the super class which are non-private.
4. All the methods declared in super interface must be implemented with public modifier in sub class.

1.4 Benefits of Inheritance

1. Enhances super class.
2. Saves time and memory.
3. Reduces coding length.
4. Increases reliability of code.
5. Allows reusability of code.

1.5 Interface

It is similar to a class, but contains only public abstract methods. Variable declarations are not allowed in an interface. Method declarations are allowed, and the implementation of the declared methods must be given in the sub class, otherwise the subclass will also be treated as an interface.

It is not possible to create an object for an interface directly. With the help of the object of the sub class, we can access all the methods of the interface.

```
interface <interface-name>
{
        // constant variable declaration
        // method declarations only (no definition)
}
```

1.6 Types of Inheritance

There are two types of inheritance:

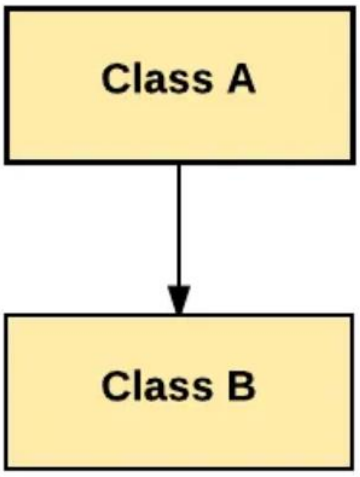
1. Classical form of inheritance
2. Containment form of inheritance

1.7 Classical Inheritance

Classical inheritance can be classified as five types, which are

1	Single Inheritance	One super class and one sub class.
2	Multiple Inheritance	Many super classes and one sub class.
3	Multilevel Inheritance	A sub class derived from another sub class.
4	Hierarchical Inheritance	One super class and many sub classes.
5	Hybrid Inheritance	A combination of different inheritance types.

1.7.1 Single Inheritance



Source Code

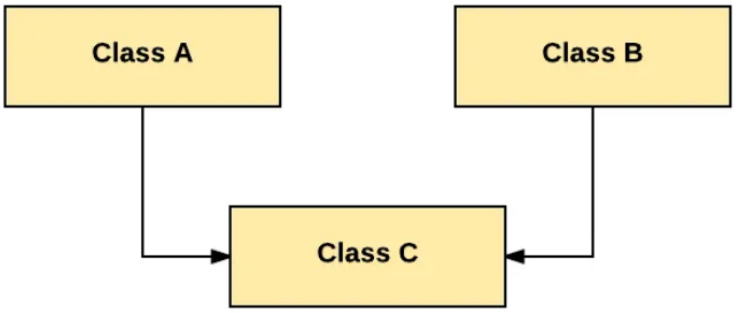
```
using k = System.Console;
class SmartDevice
{
    string name;
    double price;
    int battery;
    public SmartDevice (string name, double price, int battery)
    {
        this.name = name;
        this.price = price;
        this.battery = battery;
    }
    public void disp()
    {
        k.WriteLine("Name: " + name);
        k.WriteLine("Price: $" + price);
        k.WriteLine("Battery: " + battery + "mAh");
    }
}

class Laptop: SmartDevice
{
    string keyboard;
```

```
    int webcam;
    int ports;

    public Laptop (string name, double price, int battery, string keyboard, int
webcam, int ports) : base(name, price, battery)
    {

        this.keyboard = keyboard;
        this.webcam = webcam;
        this.ports = ports;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Keyboard Type: " + keyboard);
        k.WriteLine("Webcam Specs: " + webcam + "MP");
        k.WriteLine("No. of Ports: " + ports);
    }
}

class Program
{
    static void Main()
    {
        Laptop l = new Laptop("Dell G15", 800, 5000, "Scissor Switches", 2, 7);
        l.disp();
    }
}
```

Output

```
Name: Dell G15
Price: $800
Battery: 5000mAh
Keyboard Type: Scissor Switches
Webcam Specs: 2MP
No. of Ports: 7
```

1.7.2 Multiple Inheritance



Source Code

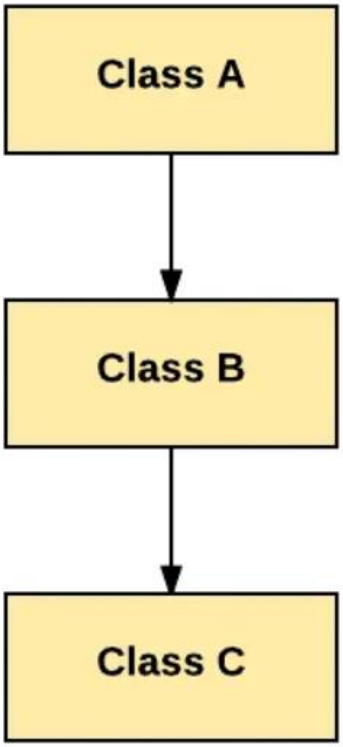
```
using k = System.Console;

interface Watch
{
    public void disp();
}
class SmartDevice
{
    string name;
    double price;
    int battery;
    public SmartDevice (string name, double price, int battery)
    {
        this.name = name;
        this.price = price;
        this.battery = battery;
    }
    public void disp()
    {
        k.WriteLine("Name: " + name);
        k.WriteLine("Price: $" + price);
        k.WriteLine("Battery: " + battery + "mAh");
    }
}

class SmartWatch: SmartDevice, Watch
{
```

```
    int size, buttons, modes;
    public SmartWatch(string name, double price, int battery, int size, int buttons,
int modes) : base(name, price, battery)
    {
        this.size = size;
        this.buttons = buttons;
        this.modes = modes;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Size: " + size + "mm");
        k.WriteLine("No. of Buttons: " + buttons);
        k.WriteLine("No. of Sports Modes: " + modes);
    }
}

class Program
{
    static void Main()
    {
        SmartWatch sw = new SmartWatch("AmazeFit", 50, 400, 43, 4, 100);
        sw.disp();
    }
}
```

Output

```
Name: AmazeFit
Price: $50
Battery: 400mAh
Size: 43mm
No. of Buttons: 4
No. of Sports Modes: 100
```

1.7.3 Multilevel Inheritance



Source Code

```
using k = System.Console;

class SmartDevice
{
    string name;
    double price;
    int battery;
    public SmartDevice (string name, double price, int battery)
    {
        this.name = name;
        this.price = price;
        this.battery = battery;
    }
    public void disp()
    {
        k.WriteLine("Name: " + name);
        k.WriteLine("Price: $" + price);
```

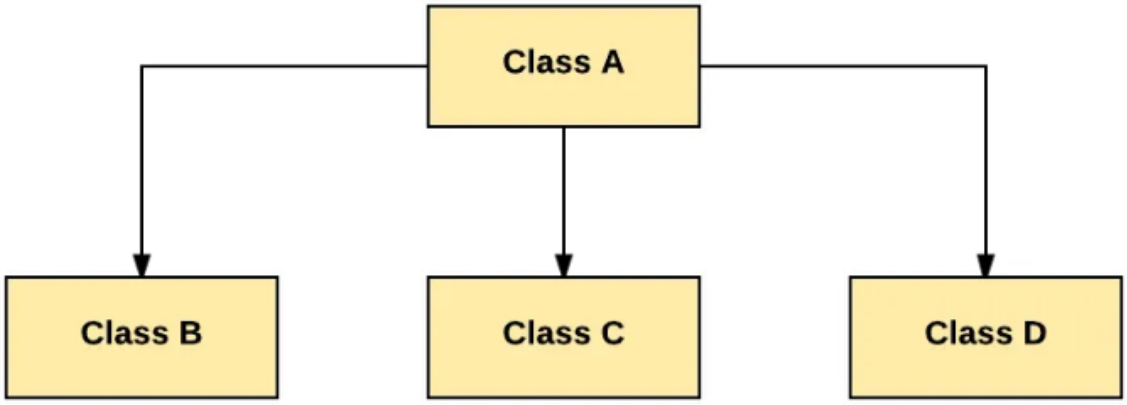
```
        k.WriteLine("Battery: " + battery + "mAh");
    }
}

class Phone: SmartDevice
{
    int speakers, cameras;
    public Phone (string name, double price, int battery, int speakers, int
cameras) : base(name, price, battery)
    {
        this.speakers = speakers;
        this.cameras = cameras;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("No. of Speakers: " + speakers);
        k.WriteLine("No. of Cameras: " + cameras);
    }
}

class Samsung: Phone
{
    string type;
    public Samsung (string name, double price, int battery, int speakers, int
cameras, string type) : base(name, price, battery, speakers, cameras)
    {
        this.type = type;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Phone Type: " + type);
    }
}

class Program
{
    static void Main()
    {
```

```
        Samsung sw = new Samsung("Samsung Galaxy ZFold 5", 1800, 5000, 4,
5, "Foldable");
        sw.disp();
    }
}
```

Output

```
Name: Samsung Galaxy ZFold 5
Price: $1800
Battery: 5000mAh
No. of Speakers: 4
No. of Cameras: 5
Phone Type: Foldable
```

1.7.4 Hierarchical Inheritance



Source Code

```
using k = System.Console;

class SmartDevice
{
    string name;
    double price;
    int battery;
    public SmartDevice (string name, double price, int battery)
    {
        this.name = name;
        this.price = price;
        this.battery = battery;
    }
    public void disp()
    {
        k.WriteLine("Name: " + name);
        k.WriteLine("Price: $" + price);
        k.WriteLine("Battery: " + battery + "mAh");
    }
}

class Phone: SmartDevice
{
    int speakers, cameras;
```

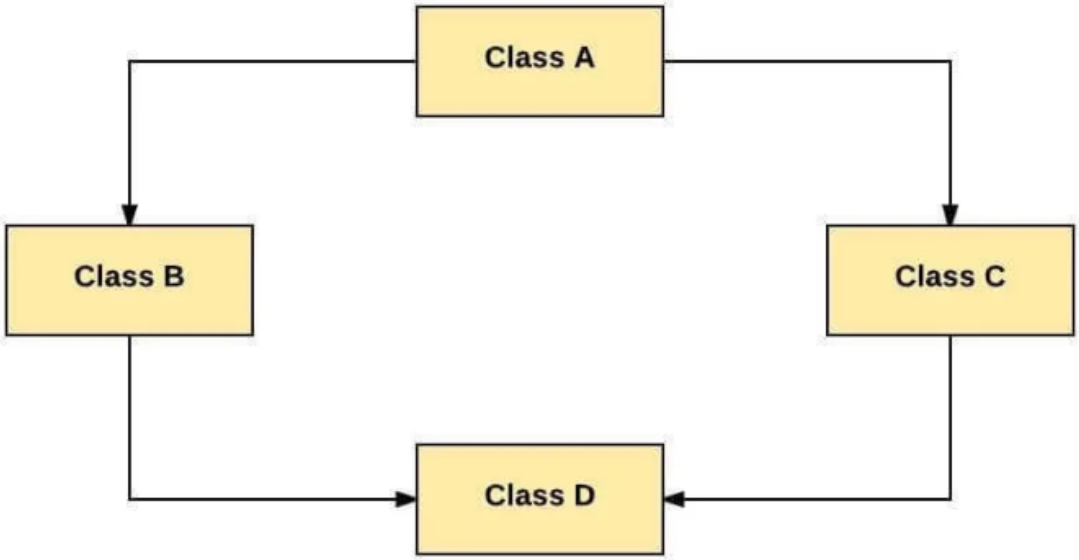
```
    public Phone (string name, double price, int battery, int speakers, int
cameras) : base(name, price, battery)
    {
        this.speakers = speakers;
        this.cameras = cameras;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("No. of Speakers: " + speakers);
        k.WriteLine("No. of Cameras: " + cameras);
    }
}
class Laptop : SmartDevice
{
    string keyboard;
    int webcam;
    int ports;

    public Laptop(string name, double price, int battery, string keyboard, int
webcam, int ports) : base(name, price, battery)
    {

        this.keyboard = keyboard;
        this.webcam = webcam;
        this.ports = ports;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Keyboard Type: " + keyboard);
        k.WriteLine("Webcam Specs: " + webcam + "MP");
        k.WriteLine("No. of Ports: " + ports);
    }
}
class Program
{
    static void Main()
    {
        Phone p = new Phone("Samsung Galaxy ZFold 5", 1800, 5000, 4, 5);
```

```
        p.disp();
        k.WriteLine();
        Laptop l = new Laptop("Dell G15", 800, 5000, "Scissor Switches", 2, 7);
        l.disp();
    }
}
```

Output

```
Name: Samsung Galaxy ZFold 5
Price: $1800
Battery: 5000mAh
No. of Speakers: 4
No. of Cameras: 5

Name: Dell G15
Price: $800
Battery: 5000mAh
Keyboard Type: Scissor Switches
Webcam Specs: 2MP
No. of Ports: 7
```

1.7.5 Hybrid Inheritance



This is a combination of hierarchical inheritance and multiple inheritance, forming a diamond structure. We can have any such combination of different inheritance types.

Source Code – Multilevel + Multiple + Hierarchical Inheritance

```
using k = System.Console;

interface Watch
{
   public void disp();
}

class SmartDevice
{
   string name;
   double price;
   int battery;
   public SmartDevice (string name, double price, int battery)
   {
      this.name = name;
      this.price = price;
      this.battery = battery;
```

```
    }
    public void disp()
    {
        k.WriteLine("Name: " + name);
        k.WriteLine("Price: $" + price);
        k.WriteLine("Battery: " + battery + "mAh");
    }
}

class SmartWatch : SmartDevice, Watch
{
    int size, buttons, modes;
    public SmartWatch(string name, double price, int battery, int size, int buttons,
int modes) : base(name, price, battery)
    {
        this.size = size;
        this.buttons = buttons;
        this.modes = modes;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Size: " + size + "mm");
        k.WriteLine("No. of Buttons: " + buttons);
        k.WriteLine("No. of Sports Modes: " + modes);
    }
}

class Samsung : SmartWatch
{
    string type,os;
    public Samsung (string name, double price, int battery, int size, int buttons,
int modes, string type, string os) : base (name, price, battery, size, buttons,
modes)
    {
        this.type = type;
    }
    public void disp()
    {
        base.disp();
```

```
        k.WriteLine("Watch Type: " + type);
        k.WriteLine("Operating System: " + os);
    }
}

class Apple : SmartWatch
{
    string band;
    int speakers;
    public Apple (string name, double price, int battery, int size, int buttons, int
modes, string band, int speakers) : base(name, price, battery, size, buttons,
modes)
    {
        this.band = band;
        this.speakers = speakers;
    }
    public void disp()
    {
        base.disp();
        k.WriteLine("Band Type: " + band);
        k.WriteLine("No. of Speakers: " + speakers);
    }
}

class Program
{
    static void Main()
    {
        Samsung s = new Samsung("Samsung Galaxy Watch Ultra", 600, 500, 48,
3, 100, "Ultra", "WearOS");
        s.disp();
        k.WriteLine();
        Apple a = new Apple("Apple Watch Ultra 2", 800, 450, 47, 2, 50,
"Rugged", 4);
        a.disp();
    }
```

Output

```
Name: Samsung Galaxy Watch Ultra
Price: $600
Battery: 500mAh
Size: 48mm
No. of Buttons: 3
No. of Sports Modes: 100
Watch Type: Ultra
Operating System:

Name: Apple Watch Ultra 2
Price: $800
Battery: 450mAh
Size: 47mm
No. of Buttons: 2
No. of Sports Modes: 50
Band Type: Rugged
No. of Speakers: 4
```

1.8 Container Inheritance

It is a type of inheritance where a class contains the object of another class. That is, a class contains another class within it through the object of this other class.

Source Code – Single Inheritance to Container Inheritance

```
using k = System.Console;

class SmartDevice
{
   string name;
   double price;
   int battery;
   public SmartDevice(string name, double price, int battery)
   {
      this.name = name;
      this.price = price;
      this.battery = battery;
   }
   public void disp()
   {
      k.WriteLine("Name: " + name);
      k.WriteLine("Price: $" + price);
      k.WriteLine("Battery: " + battery + "mAh");
   }
}

class Laptop
{
   string keyboard;
   int webcam;
   int ports;
   SmartDevice sd;
   public Laptop(string name, double price, int battery, string keyboard, int
webcam, int ports)
   {
      sd = new SmartDevice(name, price, battery);
      this.keyboard = keyboard;
      this.webcam = webcam;
```

```
        this.ports = ports;
    }
    public void disp()
    {
        sd.disp();
        k.WriteLine("Keyboard Type: " + keyboard);
        k.WriteLine("Webcam Specs: " + webcam + "MP");
        k.WriteLine("No. of Ports: " + ports);
    }
}

class Program
{
    static void Main()
    {
        Laptop l = new Laptop("Dell G15", 800, 5000, "Scissor Switches", 2, 7);
        l.disp();
    }
}
```

Output

```
Name: Dell G15
Price: $800
Battery: 5000mAh
Keyboard Type: Scissor Switches
Webcam Specs: 2MP
No. of Ports: 7
```

2 ADO .NET Database Connectivity

2.1 Introduction

ADO stands for ActiveX Data Object. It is a database framework which allows a .NET developer to work with databases, XML, and other data sources. It provides a connected and disconnected data architecture, and can be used to perform CRUD operations, as well as searching and retrieval.

Connected Architecture	**Disconnected Architecture**
Getting data from database in connection/online state.	Getting data from database in connectionless/offline state.
Holds only one table at a time.	Holds multiple tables at a time.
Can’t persist the data.	Can persist the data.
Gives faster performance.	Gives slower performance.

2.2 Connected Architecture

It has the following built-in component classes:

Component	**Description**	**Methods and Properties**
Connection	To connect to or disconnect from a database.	Open() Close()
Command	To send a command to the database. It is ready to execute an SQL query.	CommandText ExecuteReader() ExecuteNonQuery()
DataReader	To store the table results and read the records, one record into memory at a time.	Read()

2.2.1 .NET Data Providers

It is an important component of connected architecture. It supports several data providers for different types of databases.

Data Provider	Description	Required Namespace
SqlServer	To connect to Microsoft SqlServer database.	System.Data.SqlClient
ODBC	To connect to any database which supports ODBC driver.	System.Data.Odbc
OleDB	Provides support for few databases like MS Access, DB2, MySQL, etc.	System.Data.OleDb
Oracle	To connect to Oracle database.	System.Data.OracleClient

2.2.2 Sample Code and Output

Source Code

```
using System;
using y = System.Console;
using System.Data.OleDb;

namespace DBLabAccess
{
   class Program
   {
      string dsn = @"Provider=Microsoft.ACE.OLEDB.12.0;Data
Source=C:\Users\Krishna\Documents\mit.accdb";
      string tableQuery = "create table person(pid integer, pname text, cgpa
double)";
      public void insDB()
      {
         try
         {
            OleDbConnection cc = new OleDbConnection(dsn);
```

```
            cc.Open();
            OleDbCommand cd = new OleDbCommand(tableQuery, cc);
            cd.ExecuteNonQuery();
            y.WriteLine("Table Created successfully");
            cd.CommandText = "insert into person values(1,'Ganesh',9.50)";
            int r1 = cd.ExecuteNonQuery();
            cd.CommandText = "insert into person values(5,'Shiva',9.95)";
            int r2 = cd.ExecuteNonQuery();
            cd.CommandText = "insert into person values(8,'Vijay',8.90)";
            int r3 = cd.ExecuteNonQuery();
            cd.CommandText = "insert into person values(9,'Surya',7.55)";
            int r4 = cd.ExecuteNonQuery();
            cd.CommandText = "insert into person values(12,'Arul',9.25)";
            int r5 = cd.ExecuteNonQuery();
            if (r1 > 0 && r2 > 0 && r3 > 0 && r4 > 0 && r5 > 0)
            {
                y.WriteLine("Five rows are successfully inserted...");
            }
            cc.Close();
            dispDB();
        }
        catch (Exception ex)
        {
            y.WriteLine("Message: " + ex.Message);
        }
    }
    public void dispDB()
    {
        try
        {
            OleDbConnection cc = new OleDbConnection(dsn);
            cc.Open();
            OleDbCommand st = new OleDbCommand("select * from person",
cc);
            OleDbDataReader rs = st.ExecuteReader();
            y.WriteLine("\nDatabase Results");
            y.WriteLine("ID\tName\tCGPA");
            y.WriteLine("-----------------------------------------------");
            while (rs.Read())
            {
```

```
                y.WriteLine(rs[0] + "\t" + rs[1]);
            }
            y.WriteLine("-----------------------------------------------");
            cc.Close();
        }
        catch (Exception ex)
        {
            y.WriteLine("Info: " + ex.Message);
        }
    }
    static void Main()
    {
        y.WriteLine("-------------------------------------------------------");
        y.WriteLine("\t ADO.NET Connected Type - Creation, Insertion,
Retrieval");
        y.WriteLine("-------------------------------------------------------");
        Program obj = new Program();
        obj.insDB();
    }
  }
}
```

Output

```
-----------------------------------------------------------------------
         ADO.NET Connected Type - Creation, Insertion, Retrieval
-----------------------------------------------------------------------
Table Created successfully
Five rows are successfully inserted...

Database Results
ID      Name    CGPA
-----------------------------------------------
1       Ganesh  9.5
5       Shiva   9.95
8       Vijay   8.9
9       Surya   7.55
12      Arul    9.25
-----------------------------------------------
```

2.3 Disconnected Architecture

It has the following component classes:

Component	Description	Methods and Properties
DataSet	To store, modify large amount of data in a local cache view in an offline mode. Can contain multiple tables.	-
DataAdapter	Interface to copy data from the database.	Fill()
DataTable	Collection of data rows and columns.	-
DataRow	Represents data in a row.	Rows
DataColumn	Represents schema of a column.	Columns
CommandBuilder	To perform data manipulation tasks like insert, update, delete, etc.	-

2.3.1 Sample Code and Output

Source Code

```
using System;
using System.Data;
using System.Data.OleDb;
using h = System.Console;
class DisRetrieval
{
   string dsn = @"Provider=Microsoft.ACE.OLEDB.12.0;Data
Source=C:\Users\Krishna\Documents\mit.accdb";
   public void disp()
   {
      string sql = "select * from person";
      try
      {
         OleDbDataAdapter da = new OleDbDataAdapter(sql, dsn);
```

```
            DataSet ds = new DataSet();
            da.Fill(ds, "person");
            DataTable dt = ds.Tables[0];
            h.WriteLine("Retrieval via for loop");
            h.WriteLine("ID\tName\tCGPA");
            h.WriteLine("-----------------------------------------------");
            for (int i = 0; i < dt.Rows.Count; i++)
            {
                for (int j = 0; j < dt.Rows[i].ItemArray.Length; j++)
                {
                    h.Write(dt.Rows[i].ItemArray[j] + "\t");
                }
                h.WriteLine();
            }
            h.WriteLine("-----------------------------------------------");
            h.WriteLine("\nRetrieval via advanced for loop");
            h.WriteLine("ID\tName\tCGPA");
            h.WriteLine("-----------------------------------------------");
            foreach (DataRow row in dt.Rows)
            {
                h.WriteLine(row[0] + "\t" + row[1] + "\t" + row[2]);
            }
            h.WriteLine("-----------------------------------------------");
        }
        catch (Exception ex)
        {
            h.WriteLine("DB Text: " + ex.Message);
        }
    }
    static void Main()
    {
        h.WriteLine("-------------------------------------------------------------");
        h.WriteLine("\t Disconnected Architecture - Table Retrieval");
        h.WriteLine("-------------------------------------------------------------");
        DisRetrieval obj = new DisRetrieval();
        obj.disp();
    }
}
```

Output

```
-----------------------------------------------------------
         Disconnected Architecture - Table Retrieval
-----------------------------------------------------------
Retrieval via for loop
ID      Name    CGPA
-------------------------------------------------
1       Ganesh  9.5
5       Shiva   9.95
8       Vijay   8.9
9       Surya   7.55
12      Arul    9.25
-------------------------------------------------

Retrieval via advanced for loop
ID      Name    CGPA
-------------------------------------------------
1       Ganesh  9.5
5       Shiva   9.95
8       Vijay   8.9
9       Surya   7.55
12      Arul    9.25
-------------------------------------------------
```